实验三部署与测试操作手册

本手册按《实验三：MySQL 读写性能测试》的顺序组织：实验准备 → 安装 JMeter → 参数设置 → 编译与部署 → 配置 Prometheus → 执行测试 → 整理数据和报告。命令针对本组现有四台服务器，供组员重新部署和现场演示使用。长命令请从同目录的详细操作步骤.md 复制。

一 实验准备

对应指导书第 1 页。实验使用两个接口：GET /shops/{shopid}/products/{id} 查询商品，以及 POST /shops/{shopid}/products 新建商品。对 JPA、MyBatis 分别执行相同读写测试，比较响应时间、吞吐量与服务器资源使用情况。

指导书角色	本组服务器	SSH 公网地址	私网地址
A 编译和 JMeter	CDWY3	120.46.213.158	172.31.12.156
B 应用	CDWY1	120.46.82.71	172.31.1.202
C MySQL	CDWY2	120.46.146.243	172.31.8.46
D Prometheus	CDWY4	124.70.102.248	172.31.1.19

四台均为 Ubuntu 18.04.6、2 核 2 GiB、Docker 20.10.21。编译机使用 JDK 21、Maven 3.9.16、JMeter 5.6.3；数据库为 MySQL 9.2.0。服务器源码目录统一为 /root/spring-fundamentals，当前实验工程为 4. SpringAOP，包含空格，命令中必须加引号。

SSH、数据库和 Prometheus 密码由环境维护者私下提供，不写入报告。现有集群已经准备好 overlay 网络 javaee-proxy、节点标签、MySQL 数据目录、应用 secret lab3_app_properties_v1、CDWY1 私密配置 /root/.config/lab3/deploy.env，以及 CDWY4 监控认证配置。重新部署时复用这些配置。若重装系统，应先恢复这些前提，再执行本手册。

1 检查现有服务

登录 CDWY1，记录实验前状态。集群管理命令只在此节点执行。

```
ssh root@120.46.82.71
docker node ls
docker service ls
docker ps --format '{{.Names}} {{.Status}}'
```

预期：4 个节点 Ready，lab_mysql、lab_productdemoaop、lab_prometheus 各 1/1，lab_node-exporter 为 4/4。确认现场无人正在测试，再更新应用。截取节点和服务状态，保存到新建的实验记录目录。

2 备份配置和数据

CDWY1 执行以下命令，记录输出的备份路径。完整服务配置可能含敏感内容，只保存在服务器，不交给公开仓库。

```
STAMP=$(date +%Y%m%d-%H%M%S)
BACK=/root/backups/lab3-$STAMP
```

```
mkdir -p "$BACK"
chmod 700 "$BACK"
docker service inspect lab_productdemoaop > "$BACK/app-spec.json"
docker inspect demo --format '{{.State.Running}}' > "$BACK/demo-running.txt"
cp -a '/root/spring-fundamentals/4. SpringAOP/conf/docker' "$BACK/"
docker tag xmu-javaee/productdemoaop:lab3-teacher-20261018 lab3/app:before-$STAMP
printf '%s\n' "$BACK"
```

CDWY2 备份专用实验库，保留此次备份路径：

```
STAMP=$(date +%Y%m%d-%H%M%S)
BACK=/root/backups/lab3-$STAMP
mkdir -p "$BACK"
chmod 700 "$BACK"
CID=$(docker ps -q --filter name=lab_mysql | head -1)
docker exec "$CID" sh -c 'MYSQL_PWD="$MYSQL_ROOT_PASSWORD" mysqldump -uroot --single-transaction --no-tablespaces
oomall_lab3_teacher' > "$BACK/experiment.sql"
test -s "$BACK/experiment.sql"
```

二 安装和检查 JMeter

对应指导书第 2 页第 1 步。在 CDWY3 执行：

```
source /etc/profile.d/lab3-tools.sh
java -version
mvn -version
jmeter --version
```

预期显示 Java 21、Maven 3.9.16、JMeter 5.6.3。当前已安装，不需要重复安装。软件目录分别为 /opt/jdk-21、/opt/maven-current、/opt/apache-jmeter-5.6.3；每次新登录先 source 环境脚本。

如果仅 JMeter 安装目录丢失，可使用已保存的 /opt/lab3-downloads/apache-jmeter-5.6.3.tgz。指导书使用 zip 压缩包，本组保存的是相同版本的 tgz。先确认目标目录不存在，再解压；保留的 SHA-512 文件用于核对安装包。

```
cd /opt/lab3-downloads
sha512sum -c apache-jmeter-5.6.3.tgz.sha512
test ! -e /opt/apache-jmeter-5.6.3 && tar -xzf apache-jmeter-5.6.3.tgz -C /opt
source /etc/profile.d/lab3-tools.sh
jmeter --version
```

验收记录：软件版本输出截图。安装包和工具已保留在服务器，材料包提供程序及实验数据，不包含全部软件安装包。

三 设置实验参数

对应指导书第 2 页第 2 步。以下操作在 CDWY3 完成。

1 JVM 内存

先执行 free -m 查看实际内存。四台标称 2 GiB，操作系统可见内存约 1800 MiB；按实际资源设置，不使用超过机器容量的示例堆内存。

```
free -m
cd '/root/spring-fundamentals/4. SpringAOP/jmeter'
cat jmeter-command.sh
```

当前脚本中的 JVM_ARGS 为 -Xms256m -Xmx768m -Xss512k -XX:MaxMetaspaceSize=192m。直接调用 jmeter 时可设置相同 JVM_ARGS。应用 JVM 在部署文件中设置为堆 256–512 MiB，容器内存上限为 1 GiB。

2 HTML 报告时间粒度

检查 /opt/apache-jmeter-5.6.3/bin/user.properties 中是否设置以下内容；若没有，先备份该文件，再添加或修改为此值：

```
jmeter.reportgenerator.overall_granularity=1000
```

直接生成 HTML 时也可传 -Jjmeter.reportgenerator.overall_granularity=1000。该设置控制报告图表粒度，不改变服务器监控的抓取周期。

3 测试计划地址和参数

用编辑器检查 ReadProduct.jmx、WriteProduct.jmx 的用户变量 hostname=172.31.1.202、port=8080。读取计划使用同目录的 products.csv；运行时保留 CSV 与 JMX 的相对位置。不要随意更改随机写载荷和业务断言。

本组适配文件已统一使用 thread.num、rampup.period、loop.count；脚本通过 -J 传入 JMeter 属性。保留读取 HTTP 200 且 errno=0、写入 HTTP 201 的断言，结果输出到本次目录。验收记录：JVM 参数、地址和线程参数截图。

四 编译和部署应用

对应指导书第 3 页第 3 步。先在 CDWY3 编译，再把镜像传到 CDWY1 部署。

1 安装依赖并构建镜像

```
source /etc/profile.d/lab3-tools.sh
export MAVEN_OPTS='-Xmx640m -XX:MaxMetaspaceSize=256m'
cd /root/spring-fundamentals
mvn -B -ntp -f clonefactory/pom.xml -Dmaven.compiler.release=21 -Dmaven.test.skip=true install
mvn -B -ntp -f '4. SpringAOP/pom.xml' -Djava.version=21 -pl core -am -Dmaven.test.skip=true install
mvn -B -ntp -f '4. SpringAOP/pom.xml' -Djava.version=21 -Dmaven.test.skip=true package
docker build --build-arg JAR_FILE=target/productdemoaop-0.0.1-SNAPSHOT.jar -t xmu-javaee/productdemoaop:lab3-
teacher-20261018 '4. SpringAOP/productdemoaop'
```

上述命令已封装到 deploy/lab3/build-teacher.sh，也可直接运行该脚本。每步必须成功后再执行下一步。指导书用 pre-integration-test 打包镜像；本工程该阶段还会删除旧镜像，因此这里用 package 加 docker build 完成打包，保留旧镜像用于回退。截取 BUILD SUCCESS 和镜像构建结果。

2 传输镜像

CDWY3 导出镜像，通过本机 SFTP 下载后上传 CDWY1。服务器之间不假设免密登录。

```
# CDWY3
docker save xmu-javaee/productdemoaop:lab3-teacher-20261018 | gzip > /root/lab3-app.tar.gz
sha256sum /root/lab3-app.tar.gz
# After SFTP transfer, on CDWY1
sha256sum /root/lab3-app.tar.gz
gzip -dc /root/lab3-app.tar.gz | docker load
docker image inspect xmu-javaee/productdemoaop:lab3-teacher-20261018 --format '{{.Id}}'
```

两端归档文件哈希及镜像 ID 应一致。重新构建可能产生不同于旧镜像的 ID，这是正常现象；应比较本轮两台机器的结果。

3 检查实验数据库

在 CDWY2 执行：

```
CID=$(docker ps -q --filter name=lab_mysql | head -1)
docker exec -i "$CID" sh -c 'MYSQL_PWD="$MYSQL_ROOT_PASSWORD" mysql -uroot -N oomall_lab3_teacher' <<'SQL'
SELECT COUNT(*),MAX(id) FROM goods_product;
SELECT COUNT(*) FROM goods_goods;
```

```
SELECT COUNT(*) FROM goods_onsale;
SELECT AUTO_INCREMENT FROM information_schema.TABLES WHERE TABLE_SCHEMA='oomall_lab3_teacher' AND
TABLE_NAME='goods_product';
SQL
```

预期依次为 3912/5461、30660、3914、5609。专用库名中的 teacher 是已有数据库名称，命令中保持不变。原运行库 oomall_demo 当前商品数为 3913，不用于正式性能测试。

重新部署不需要重新初始化 MySQL。若专用库确实丢失，先恢复实验前备份；初次创建空库才使用材料中的 data/teacher-product.sql.gz，并给应用账号授权。不要覆盖原运行库或删除 MySQL 数据目录。保存初始行数截图。

4 部署 JPA 与 MyBatis

在 CDWY1 执行：

```
cd /root/spring-fundamentals
PERSISTENCE=jpa LAB_DATABASE=oomall_lab3_teacher LAB3_BLOCK=check-jpa bash deploy/lab3/deploy-teacher.sh
docker service ls
docker service ps lab_productdemoaop
curl -fsS http://127.0.0.1:8080/actuator/health
curl -fsS http://127.0.0.1:8080/actuator/prometheus | grep '^jvm_threads_live_threads'
```

等待健康 UP，确认 persistence="jpa"。如果重新构建覆盖了同名镜像，而服务没有更换容器，在上述部署后执行 docker service update --force lab_productdemoaop，等待健康并核对实际容器镜像 ID。

完成 JPA 测试后，用同一入口切换 MyBatis：

```
PERSISTENCE=mybatis LAB_DATABASE=oomall_lab3_teacher LAB3_BLOCK=check-mybatis bash deploy/lab3/deploy-teacher.sh
```

等待健康 UP、persistence="mybatis" 后再测试。唯一部署文件为 4. SpringAOP/conf/docker/productdemoaop-docker.yml，使用现有 Docker secret 注入应用密码。两实现共用接口、业务层和数据集，仅切换持久层。Tomcat 最大 300/最小 100，Druid 最大 300/最小 100/初始 100，保留 SQL 和业务日志设置。保存两套实现的健康状态截图。

五 配置和检查 Prometheus

对应指导书第 3 页第 4 步。源码配置位于 4. SpringAOP/conf/prometheus，运行配置位于 CDWY4 的 /root/prometheus。需要修改时先备份运行配置，通过 SFTP 更新相应文件；保留 web.yml 和 self-scrape-password 中的现有认证设置。

1 检查监控目标

node 任务的四个目标应为 172.31.1.202:9100、172.31.8.46:9100、172.31.12.156:9100、172.31.1.19:9100。应用目标为 productdemoaop:8080，另有 Prometheus 自身目标。修改运行配置后，在 CDWY4 检查语法：

```
CID=$(docker ps -q --filter name=lab_prometheus | head -1)
docker exec "$CID" promtool check config /etc/prometheus/prometheus.yml
```

只有确实修改配置时，才到 CDWY1 执行 docker service update --force lab_prometheus 使其重启加载，然后核查目标和规则。现有配置正确时无需重复修改。

2 浏览器查看和截图

在本机建立隧道，浏览器打开 http://127.0.0.1:19091，使用本组 Prometheus 密码登录：

```
ssh -N -L 19091:127.0.0.1:9090 root@124.70.102.248
```

Targets 页面应有 6 个目标全部 UP；Rules 页面应有 13 条规则且没有错误。按指导书依次查询：

```
spring:jvm_threads_live_threads
node:cpu_usage_percent
node:memory_usage_percent
```

分别记录活跃线程、CPU 和内存曲线，时间窗口覆盖测试起止。抓取周期为 2 秒，全局 evaluation 为 5 秒，两个规则组实际每 30 秒评估。CPU 为 rate[5m] 平滑值，不作为十秒测试的瞬时峰值；内存口径使用 Free+Buffers+Cached。保存 Targets、Rules 和三个查询画面。

六 执行读写性能测试

对应指导书第 3 页第 5 步。正式采集前先确认应用 profile、专用库初始行数和监控健康。在 CDWY1 记录 demo 原运行状态后执行 docker stop demo，避免与压测争抢资源；结束后按原状态恢复。

1 按测试计划运行一组测试

以下命令在 CDWY3 执行。先将测试计划及 CSV 复制到全新目录，以免覆盖旧结果。PROFILE 按当前应用实际实现填写 jpa 或 mybatis，REP 为重复轮次。

```
source /etc/profile.d/lab3-tools.sh
SRC='/root/spring-fundamentals/4. SpringAOP/jmeter'
BATCH=/root/lab3-results/check-$(date +%Y%m%d-%H%M%S)
PROFILE=jpa
REP=1
WORK="$BATCH/$PROFILE-r$REP"
mkdir -p "$WORK"
cp "$SRC/ReadProduct.jmx" "$SRC/WriteProduct.jmx" "$SRC/products.csv" "$SRC/name.csv" "$SRC/jmeter-command.sh"
"$WORK/"
cd "$WORK"
date -Is > read-start.txt
sh jmeter-command.sh ReadProduct 1000 10 2 > read-console.log 2>&1
date -Is > read-end.txt
tail -10 read-console.log
```

参数依次是计划名、总线程数、升压秒数、循环次数。预期生成 ReadProduct-1000-10-2.jtl 和 ReadProduct-1000-10-2/index.html，记录 2000 次请求，业务断言失败 0。下载整个 HTML 目录后打开 index.html，不能只下载单个 HTML 文件。保存终端汇总与 Statistics、响应时间、吞吐量页面截图。

读测试后再次确认数据库仍为 3912 条商品，再执行写测试：

```
cd "$WORK"
# Preserve the previous JMeter diagnostic log
if [ -f jmeter.log ]; then mv jmeter.log read-jmeter.log; fi
date -Is > write-start.txt
sh jmeter-command.sh WriteProduct 1000 10 1 > write-console.log 2>&1
date -Is > write-end.txt
tail -10 write-console.log
```

预期生成 WriteProduct-1000-10-1.jtl 和对应 HTML 目录。写入成功 1000 次时，商品应增至 4912 条。记录实际成功数与新增行数是否一致。归档时按实现、操作和参数命名 JTL，保留原始内容。指导书的 1000-10-1 是运行示例；本次读取按参考样例使用 1000-10-2，写入使用 1000-10-1。

2 每组之间复位数据

写测试后先保存行数证据，再在 CDWY2 对已经备份的专用库执行下列命令。只适用于本实验仅新增 id>5461 商品的情况：

```
CID=$(docker ps -q --filter name=lab_mysql | head -1)
docker exec -i "$CID" sh -c 'MYSQL_PWD="$MYSQL_ROOT_PASSWORD" mysql -uroot -N oomall_lab3_teacher' <<'SQL'
DELETE FROM goods_product WHERE id>5461;
ALTER TABLE goods_product AUTO_INCREMENT=5609;
```

```
SELECT COUNT(*),MAX(id) FROM goods_product;
SQL
```

预期恢复 3912/5461。每轮都检查初始条件。若业务程序更新或删除了原始行，应从专用库备份恢复，不能仅删除新增行。

3 完成要求的对比测试

操作	总线程数	升压	循环	实现	提交组数
读取	1000、1050、1100、1150、1200	10 秒	2	JPA、MyBatis	10
写入	1000、1100、1200、1300	10 秒	1	JPA、MyBatis	8

按上表修改 jmeter-command.sh 的线程参数。每次新建 WORK 目录，切换实现先按第四节部署并等待健康；每组前后核对数据库行数，写入后按上一节复位。两种实现使用相同接口和参数，由此完成 JPA 性能测试、MyBatis 读写比较及框架对比。

本次提交统一保留已采集的第一轮，共 18 组 31200 次请求。读取共 22000 次，写入共 9200 次；不再纳入 100/400 线程或第二、第三轮。现场重新演示应保存到新目录，不覆盖提交数据。

4 保存测试数据和监控

每组保存 JMeter 生成的原始 JTL、测试参数和数据库前后行数。按该组起止时间导出第五节的三项 Prometheus 查询，窗口覆盖测试前后。通过 query_range 接口保存 JSON，例如在已建立隧道的本机执行：

```
curl -u admin -G 'http://127.0.0.1:19091/api/v1/query_range' --data-urlencode 'query=node:cpu_usage_percent' --data-urlencode 'start=START' --data-urlencode 'end=END' --data-urlencode 'step=2' -o cpu.json
```

START、END 替换为实际 Unix 秒时间戳。对内存、活跃线程分别导出；Windows PowerShell 使用 curl.exe。材料包将同组的三项原始查询结果和测试参数放在一个中文监控 JSON 中，便于对应 JTL。

保存 JMeter 测试报告截图；环境配置截图另存。网页报告保留整个目录，整理时合成一个 ZIP，查看前整体解压。

5 结束并恢复服务

先保存全部数据、日志和截图，再复位专用库。CDWY1 按实验开始前的记录恢复服务；本组正常运行配置为 jpa/oomall_demo：

```
cd /root/spring-fundamentals
PERSISTENCE=jpa LAB_DATABASE=oomall_demo LAB3_BLOCK=restored bash deploy/lab3/deploy-teacher.sh
# Run only if demo was running before the experiment
docker start demo
docker service ls
curl -fsS http://127.0.0.1:8080/actuator/health
```

检查专用库 3912 条、原运行库 3913 条、应用 UP、监控 6 目标 UP、13 条规则 OK，demo 回到原状态，保存截图。若本次构建无法启动，可用 APP_IMAGE 指定第一节保存的 lab3/app:before-时间戳，恢复备份配置后重新部署。异常退出也要完成恢复检查。

七 验收展示与数据说明

对应指导书第 4 页：提交实验程序 Git 地址、原始 JTL 和实验报告。材料包根目录 README 列明每个目录对应的实验内容。

1 验收展示顺序

展示内容	打开什么	核对什么
环境与部署	四节点及服务状态；过程截图/环境配置	编译、应用、MySQL、监控分工，服务正常
JPA 性能测试	原始数据/01_JPA 读取、02_JPA 写入	读取 5 组、写入 4 组的原始 JTL
MyBatis 读写对比	原始数据/03_MyBatis 读取、04_MyBatis 写入；数据统计	相同线程档位的读写结果；与 JPA 相同操作对比
JMeter 测试结果	解压 JMeter 测试报告.zip，打开对应组 index.html	请求数、错误数、响应时间统计与趋势
运行监控	原始数据/05_Prometheus 监控	对应组的 JVM 活跃线程、CPU 和内存查询结果
程序和报告	实验报告；程序 Git 地址	代码、实验设计、结果和图表相互对应

程序地址：https://github.com/Re1lya/lab3-mysql-benchmark

2 数据各代表什么

数据或字段	含义
JTL 的一行	一次 HTTP 请求的记录
timeStamp	请求发起时间，Unix 毫秒时间戳
elapsed	完整响应耗时，毫秒；报告均值与百分比使用它
Connect	JMeter 记录的连接建立耗时，毫秒
responseCode 与 success	HTTP 响应码与测试计划断言是否通过
平均响应时间	该组全部请求 elapsed 的平均值
P95	约 95%的请求耗时不超过该数值
40ms 内成功比例	成功且 elapsed≤40ms 的请求数÷全部请求数
JVM 活跃线程	应用进程中存活的线程数
CPU、内存使用率	按配置的 Prometheus 规则计算的节点资源指标

线程数×循环次数等于本计划的预期请求数。例如 1200-10-2 读取为 2400 次，1300-10-1 写入为 1300 次。对比同一操作时两种实现参数一致。样例的读循环为 2、写循环为 1，查看读写比较时按表中实际参数理解。

原始数据/测试组索引.csv 对应每组的 JTL、监控文件和网页报告目录。JTL 由 JMeter 直接生成；监控来自 Prometheus 接口；数据统计中的 CSV 由原始记录计算。原始 JTL 只读保存，不在 Excel 中另存覆盖。

3 现场演示一组测试

先确认当前 profile 及初始商品 3912 行，再按第六节运行一组读取或写入。展示终端汇总、新生成的 JTL 和 HTML 报告，以及同一时间段的 Prometheus 查询。写入后核对新增行数，复位数据并恢复实验前服务状态。现场数据使用新日期目录，既有报告数据仍保留原采集日期。